

杂项

Morning Flower and Evening Oath

XCPC vector 模板分册

目录

使用约定	ii
第一章 离线算法	1
1.1 莫队算法	1
1.1.1 带修改莫队	1
1.1.1.1 带修改的莫队	1
1.1.2 莫队二次离线	3
1.1.2.1 莫队二次离线：异或位数配对	3
第二章 配套知识与验证范围	6
2.1 带修莫队：时间轴与区间状态	6
2.2 莫队二次离线：前缀事件与自身扣除	7

使用约定

C++20，默认包含 `bits/stdc++.h` 与 `cassert`，并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

第一章 离线算法

1.1 莫队算法

1.1.1 带修改莫队

1.1.1.1 带修改的莫队

ModifiedMo(n, m) 保存离线询问, $n > 0$, m 为修改总数。add(l, r, t) 登记第 t 次修改完成后的 $[l, r]$, 返回从 0 开始的询问编号; $0 \leq l \leq r \leq n$, $0 \leq t \leq m$, 询问数小于 INT_MAX。run(add, del, chg, ans, block=0) 排序并移动三个指针。add(p)/del(p) 按下标更新当前区间统计; chg($i, forward, l, r$) 将编号 i 的修改前进或回退, 并同步修正当前 $[l, r]$ 内的贡献; ans(id) 记录结果。修改编号从 0 开始, 前进第 i 次修改时原时间为 i , 回退时原时间为 $i+1$ 。回调初始状态必须为原数组、空区间、时间 0; 重复 run 前由调用方重置状态。状态不会自动恢复, 原询问顺序不变。仅适用于可增删维护的区间统计, 需区分左右端操作时另行拆分回调。默认块长约 $n^{2/3}$, 也可显式指定正块长。 n 、询问数、修改数同阶且回调 $O(1)$ 时总时间 $O(n^{5/3})$, 调度器空间 $O(q)$ 。

```
1 struct ModifiedMo
2 {
3     struct Query
4     {
5         int l, r, t, id;
6     };
7
8     int n, m;
9     vector<Query> q;
10
11     ModifiedMo(int n, int m) : n(n), m(m) { assert(n > 0 && m >= 0); }
12
13     int add(int l, int r, int t)
14     {
15         assert(0 <= l && l <= r && r <= n);
16         assert(0 <= t && t <= m);
17         int id = q.size();
18         q.push_back({l, r, t, id});
19         return id;
20     }
```

```
22     template <class Add, class Del, class Chg, class Ans>
23     void run(Add add, Del del, Chg chg, Ans ans, int block = 0) const
24     {
25         if (q.empty()) return;
26         if (block == 0) block = max(1, int(pow(n, 2.0 / 3)));
27         assert(block > 0);
28         auto ord = q;
29         auto cmp = [&](const Query &a, const Query &b)
30         {
31             int x = a.l / block, y = b.l / block;
32             if (x != y) return x < y;
33             int p = a.r / block, q = b.r / block;
34             if (p != q) return (x & 1) ? p > q : p < q;
35             return ((x ^ p) & 1) ? a.t > b.t : a.t < b.t;
36         };
37         sort(ord.begin(), ord.end(), cmp);
38         int l = 0, r = 0, t = 0;
39         for (auto q : ord)
40         {
41             while (t < q.t)
42             {
43                 chg(t, true, l, r);
44                 t++;
45             }
46             while (t > q.t)
47             {
48                 t--;
49                 chg(t, false, l, r);
50             }
51             while (l > q.l) add(--l);
52             while (r < q.r) add(r++);
53             while (l < q.l) del(l++);
54             while (r > q.r) del(--r);
55             ans(q.id);
56         }
57     }
58 };
```

1.1.2 莫队二次离线

1.1.2.1 莫队二次离线：异或位数配对

`xor_hamming_pairs(a, queries, k, bits=14)` 按查询输入顺序返回 long long 答案, 统计 $1 \leq i < j < r$ 且 $\text{popcount}(a[i] \oplus a[j]) = k$ 的下标对。queries 为 0-based 半开区间, 允许空区间、重复查询与空数组; $0 \leq \text{bits} \leq 20$, $0 \leq a[i] < 2^{\text{bits}}$, n , m 和容器下标须可由 int 表示。k 超出 $[0, \text{bits}]$ 直接返回全零; 相等值来自不同下标才构成一对。莫队移动生成前缀事件, 第二次离线按前缀扫描; $k=0$ 显式扣掉自身。令 $C = \text{binom}(\text{bits}, k)$, 时间 $O(2^{\text{bits}} + nC + m \log(m+1) + n\sqrt{m+m})$, 空间 $O(2^{\text{bits}} + n + m)$, $m=0$ 直接返回。无需维护可逆的在线窗口, 也不支持修改。

```

1 // Queries are 0-based half-open; count unordered pairs of distinct indices.
2 vector<long long> xor_hamming_pairs(const vector<int> &a,
3                                     const vector<pair<int, int>> &queries,
4                                     int k,
5                                     int bits = 14)
6 {
7     int n = a.size(), m = queries.size();
8     vector<long long> answer(m);
9     assert(0 <= bits && bits <= 20);
10    int universe = 1 << bits;
11    for (int x : a) assert(0 <= x && x < universe);
12    for (auto [l, r] : queries) assert(0 <= l && l <= r && r <= n);
13    if (!m || k < 0 || k > bits) return answer;
14    vector<int> masks;
15    for (int x = 0; x < universe; x++)
16        if (popcount(unsigned(x)) == k) masks.push_back(x);
17    vector<int> count(universe);
18    vector<long long> prefix(n + 1);
19    for (int i = 0; i < n; i++)
20    {
21        prefix[i + 1] = prefix[i] + count[a[i]];
22        for (int mask : masks) count[a[i] ^ mask]++;
23    }
24    int block = max(1, int(n / sqrt(m)));
25    vector<int> order(m);
26    iota(order.begin(), order.end(), 0);
27    sort(order.begin(),
28         order.end(),
29         [&](int x, int y)
30         {
31             int bx = queries[x].first / block, by = queries[y].first / block;
32             if (bx != by) return bx < by;
33             return bx & 1 ? queries[x].second > queries[y].second
34                          : queries[x].second < queries[y].second;
35         });
36
37    struct Event
38    {
39        int id, l, r, sign;
40    };
41
42    vector<vector<Event>> events(n + 1);

```

莫队端点移动与前缀事件（接上页同一函数）：

```
43     int l = 0, r = 0;
44     for (int id : order)
45     {
46         auto [ql, qr] = queries[id];
47         if (r < qr)
48         {
49             events[l].push_back({id, r, qr, -1});
50             answer[id] += prefix[qr] - prefix[r];
51             r = qr;
52         }
53         if (l > ql)
54         {
55             events[r].push_back({id, ql, l, 1});
56             answer[id] -= prefix[l] - prefix[ql];
57             l = ql;
58         }
59         if (r > qr)
60         {
61             events[l].push_back({id, qr, r, 1});
62             answer[id] -= prefix[r] - prefix[qr];
63             r = qr;
64         }
65         if (l < ql)
66         {
67             events[r].push_back({id, l, ql, -1});
68             answer[id] += prefix[ql] - prefix[l];
69             l = ql;
70         }
71     }
```

第二次离线与差分还原（接上页同一函数）：

```

72     fill(count.begin(), count.end(), 0);
73     for (int p = 0; p <= n; p++)
74     {
75         if (p)
76             for (int mask : masks) count[a[p - 1] ^ mask]++;
77         for (auto e : events[p])
78             for (int i = e.l; i < e.r; i++)
79             {
80                 int value = count[a[i]] - (k == 0 && i < p);
81                 answer[e.id] += 1LL * e.sign * value;
82             }
83     }
84     long long total = 0;
85     for (int id : order)
86     {
87         total += answer[id];
88         answer[id] = total;
89     }
90     return answer;
91 }

```

模板题使用：Luogu P4887

题意：统计每个区间内异或值恰有 k 个 1 的无序下标对。输入 1-based 闭区间 $[l, r]$ ，只将 1 减一转为 0-based 半开区间； k 可以大于 14，此时全零。 $k=0$ 统计相等值的不同下标，不计自身。

所需模板：xor_hamming_pairs。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P4887>

```

1  int main()
2  {
3      ios::sync_with_stdio(false);
4      cin.tie(nullptr);
5      int n, m, k;
6      cin >> n >> m >> k;
7      vector<int> a(n);
8      for (int &x : a) cin >> x;
9      vector<pair<int, int>> queries(m);
10     for (auto &[l, r] : queries)
11     {
12         cin >> l >> r;
13         l--;
14     }
15     for (long long x : xor_hamming_pairs(a, queries, k)) cout << x << '\n';
16 }

```

第二章 配套知识与验证范围

2.1 带修莫队：时间轴与区间状态

先离线读完操作，记录每个单点修改的下标、修改前值和修改后值。询问的时间 t 表示前 t 次修改已执行；输入中的询问本身不推进时间。同一位置的第二次修改，其旧值必须来自第一次修改后的数组，不能始终使用初始值。需要离散化时，应将全部初值与修改后值一同纳入。

调度器保持三个指针 (l, r, t) ，回调统计恰好对应当前版本的半开区间 $[l, r)$ 。前进修改 i 时写入 after，回退时写入 before。若修改点在当前区间内，必须先删除旧贡献、改值、再加入新贡献；区间外只改数组。随后移动区间端点，最后按原询问编号写答案。扩展区间先于收缩，支持空区间和互不相交区间。整个 run 结束后状态停留在排序后的最后一条询问；再次调用前须重置原数组和统计量。

左端点分块、右端点分块，再按时间排序，相邻块采用蛇形顺序。时间关键字是真实修改次数，不能把询问编号当作时间维度。令块长为 B ，询问数为 Q ，修改数为 U ，则常数时间回调下的移动量上界为

$$O\left(QB + \frac{n^2}{B} + U \min(Q, (\lfloor n/B \rfloor + 1)^2)\right).$$

另有 $O(Q \log Q)$ 排序与 $O(Q)$ 调度空间。三者同阶时取 $B \approx n^{2/3}$ ，得到 $O(n^{5/3})$ ；规模悬殊时可手动调整块长。回调需要区分左右端增删时应拆分四个方向，当前简版适合不同数、频次矩等统计。

P1903 数颜色 驱动以颜色频次数组维护不同数，真实操作顺序建立修改前后值，随后交给调度器。本地快照测试逐项检查每个时间、每个区间及修改回退状态，另以顺序 set 查询核对完整驱动。最大规模验证 133333 个元素和操作，并覆盖仅修改、仅询问与重复修改同一位置。最终简短命名版本在 **P1903 记录 297653743** 通过全部 13 个测试，最慢单点 546 ms、峰值内存 14.65 MB。此记录使用默认块长与不同颜色数统计；自定义块长、空区间及任意登记时间由本地测试覆盖。最快提交榜的排名及总数尚未核验。

2.2 莫队二次离线：前缀事件与自身扣除

对区间内满足 $H(i, j) = [\text{popcount}(a_i \text{ xor } a_j) = k]$ 的无序下标对，使用 `xor_hamming_pairs`。令 $t(i) = \sum_{j < i} H(i, j)$, $P(x) = \sum_{i < x} t(i)$, 以及 $T(p, i) = \sum_{j < p} H(i, j) - [k = 0 \wedge i < p]$ 。右端扩展时贡献为 $t(i) - T(L, i)$, 左端扩展为 $T(R, i) - t(i)$; 删除取相反数。整段端点移动的 t 和由 P 的区间差求出, 剩余部分存成按前缀长度分桶的事件。第二遍顺扫只做插入, 每次枚举异或后有 k 个置位的掩码更新计数; 事件中的每个下标只读一次计数, 并在 $k = 0$ 时扣自身。最后按莫队顺序累加窗口答案差, 再按原编号返回。

区间统一为 0-based 半开区间, 先扩右、扩左, 再缩右、缩左, 保持窗口合法。允许空区间; 没有查询时直接返回。位宽默认为 14, 若 k 超出位宽则答案全零。P4887 的 $k < 2^{14}$ 并不代表 $k \leq 14$ 。令 $C = \binom{b}{k}$, 块长约为 $n/\sqrt{m}$, 总时间 $O(2^b + nC + m \log(m+1) + n\sqrt{m} + m)$, 空间 $O(2^b + n + m)$ 。端点移动不再乘枚举掩码数 C 。答案与前缀和使用 long long; 十万个相等数的配对数超过 int32。本地全枚举、随机逐对参考及十万规模的 $k = 0, 7$ 分支通过; 线上验证待补。

算法索引

ModifiedMo, 1

xor_hamming_pairs, 3

带修莫队, 6